Brief del proyecto: Vemo · Business Case Growth Analyst

Qué es: cómo resolví el business case de Growth Analyst de Vemo, de los Excel a la recomendación. Cubre qué hice en cada paso, con qué herramienta, qué decidí y qué verifiqué.

Tres decisiones de Dirección:

- **Impulso:** ¿llegamos a 400 entregas en un mes?
- **Conduce:** ¿Marketing, Operaciones u otra cosa con $1.5 M?
- **Watts:** ¿repetimos la promo de agosto?

Respuestas:

Case	Respuesta	Pesos
Impulso	No es viable el próximo mes; realista en **marzo 2027**	+$6.1 M/mes de margen neto en el mes de 400 (costo $2.3 M/mes)
Conduce	**Ninguna** de las dos; reclutamiento cercano en el sur + permanencia temprana	$0.9–1.7 M de contribución en Q1 con $1.07 M comprometidos
Watts	**No**; medir bien, reparar conectores, flota Vemo en horas valle, precio por horario con piso	Evita perder $0.19–0.25 M de margen en Q1
Pregunta final	Analizar a fondo **Impulso**	$23.7 M en juego en el trimestre vs $2.6 M (Conduce) y $0.2 M (Watts)

Entregable	Dónde
App (decks, dashboards, anexo)	https://vemo-business-case.vercel.app
Google Sheet del anexo	https://docs.google.com/spreadsheets/d/1FK5lK3L-FRlHqhxH8Y-Vezhyul2obZITFUhgZcq9e-A/edit
Repositorio (SQL y código)	https://github.com/Jesus105/vemo_bc
Deck de análisis (7 láminas)	`entregables/Vemo_Deck_Analisis.pdf`
Deck de uso de IA (5 láminas)	`entregables/Vemo_Deck_IA.pdf`
Anexo de cálculos	`entregables/Vemo_Anexo_Calculos.xlsx` y `entregables/Anexo_Uso_IA_y_Calculos.md`
Investigación pública	`docs/investigacion_mercado.md`

Paso 0 · Montar la base antes de analizar

Qué: tener los datos en un lugar donde se pudieran consultar con SQL, sin modificarlos.

Cómo:

1. **Exportación sin limpieza.** Exporté cada hoja de los 3 Excel a CSV con `tools/data_prep/xlsx_to_csv.py`: fechas en formato ISO, números sin redondear y vacíos como campo vacío. El conteo de filas por hoja quedó en `data/raw/manifest.csv`.

2. **Postgres 16 en Docker** (`docker-compose.yml`, puerto 5433). Creé un schema por línea de negocio (`impulso`, `conduce`, `watts`) y cargué los CSV con `sql/01_schema.sql` y `sql/02_load_data.sql`.
    - Las tablas no tienen llaves ni restricciones a propósito. Cada LEEME dice que los datos salen "tal como los exportan los sistemas", y una restricción habría rechazado o corregido filas en silencio.
3. **MCP `postgres-vemo`** (`.mcp.json`) de solo lectura, para que Claude Code consultara la base en vivo.
4. **Skill `vemo-analyst`** (`.claude/skills/vemo-analyst/`) con el contexto del caso, el modelo de datos y las reglas de trabajo:
    - no confiar en los datos a ciegas;
    - no tocar las tablas crudas: toda limpieza va en vistas;
    - escribir los supuestos;
    - toda recomendación en pesos y con horizonte;
    - no mezclar cohorte con calendario;
    - recalcular las cifras clave por una segunda vía.
5. Tomé como guía la forma de trabajar de mi skill `meli-bi-analyst`: respuesta en una frase, de 2 a 4 números de evidencia, supuestos y luego impacto. No usé sus tablas.

Resultado: 16 tablas cargadas y conteos validados contra el manifest con `scripts/verify_db.sh`.

Paso 1 · Verificar que la base es idéntica a los Excel

Qué: confirmar que lo que iba a consultar era exactamente lo que entregó Vemo.

Cómo: `scripts/verify_xlsx_vs_db.py` abre los Excel originales, no los CSV, y compara cada celda contra Postgres:

- **Números:** exactos con `Decimal`, sin tolerancia.
- **Fechas:** en formato ISO.
- **Vacíos:** deben ser NULL en la base.
- **Encabezados:** deben coincidir con las columnas de la tabla.
- **Filas:** como no hay llave, comparo el conjunto completo de filas sin importar el orden.
- **LEEME:** línea por línea y en orden.
- **`supuestos`:** además, que `valor_num` sea igual a `valor`.

Resultado: 32,740 celdas en 16 hojas, 0 diferencias.

Paso 2 · Contexto externo con investigación pública

Qué: entender las líneas de negocio y tener benchmarks para defender las recomendaciones.

Cómo: lancé un agente con Firecrawl (búsqueda y scraping) en paralelo al análisis, solo con consultas públicas y sin datos del caso. Revisó:

- vemovilidad.com y prensa sobre Vemo.
- Las primeras páginas del Reporte de Sustentabilidad.
- **Competencia en Impulso:** Moove, Kovi, DiDi Keko, OCN.
- **Churn de conductores:** Uber, transporte de carga y el efecto del tiempo de traslado.
- **Utilización de cargadores:** McKinsey y el estudio de descuentos en el Reino Unido.

Resultado: `docs/investigacion_mercado.md`. Lo que más usé:

- **Fintechs de flota:** precalifican con datos de la plataforma en lugar de revisión manual. Eso respalda el filtro previo en el agente de IA.
- **Traslado:** la probabilidad de renunciar sube con traslados de más de 30–45 min. Eso es consistente con el hallazgo de distancia en Conduce.
- **Descuentos en carga:** un 40 % de descuento sube el uso 117 %, pero la mitad es canibalización de otras redes.

Corrección: la web de Vemo reporta 141, “400+” o “1,000+” puntos de carga según la página, y requisitos de Impulso distintos a los del caso. Decidí usar la web solo como contexto y decidir con los datos del caso.

Paso 3 · Impulso: diagnóstico del funnel

Qué: encontrar de 3 a 4 insights y entender qué limita las entregas.

Cómo (queries en vivo vía MCP, luego en `sql/03_analytical_views.sql`):

1. **Funnel calendario con tasas por etapa** (`impulso.v_funnel_tasas`), separando antes y después del agente de IA (1-may):

```sql
SELECT mes, enviados_a_analisis, rechazados_riesgo, calificados,
       round(calificados::numeric / enviados_a_analisis, 3) AS tasa_aprobacion,
       enviados_a_analisis - 700 AS envios_sobre_capacidad
FROM impulso.funnel_mensual;
```

   - Contacto: 62 % → 80 %.
   - Envíos a análisis: ×2.8 (de ~290 a 812 por mes).
   - Aprobación: 53 % → 33 %.
   - Rechazo de riesgo: 15 % → 50 % de los envíos.
   - En 5 de 6 meses los envíos superaron la capacidad de Riesgo (700).
   - Días de análisis: 3.9 → 7.9.

2. **Exceso de rechazos de riesgo** (`impulso.v_exceso_riesgo`): rechazos por encima de la tasa pre-agente.

   - **Supuesto:** el caso dice que Riesgo siempre rechaza a quien no cumple los requisitos mínimos, así que ese exceso son leads sin requisitos que el agente deja pasar.
   - **Resultado: 1,692 leads en may–oct (~282/mes), $305 k en envíos**, además de saturar la capacidad.

3. **Cohorte vs calendario.** El reporte de Dirección (`funnel_mensual`) cuenta eventos del mes sin importar cuándo se creó el lead. Noviembre marca 24 entregas con 12 visitas. Por eso calculé todas las tasas con **cohortes cerradas may–sep** (`impulso.v_cohortes_tasas`):

   - Entregas por cohorte: **81 → 137 al mes (+69 %)**. El agente sí funciona.
   - Lead → entrega: 6.7 %; calificado → entrega: 48.6 %.
   - 17 % de las visitas aprobadas se pierde por el depósito.

4. **Canales** (`impulso.v_canales_cac`, cohortes may–sep):

   - CAC: Meta $2.7 k, Google $6.2 k, TikTok $1.7 k; total $2.9 k contra un tope de $10 k.
   - La inversión cayó 33 % de junio a octubre ($453 k → $304 k) y los leads bajaron de 2,201 a 1,613. Hoy el freno es la demanda.

5. **Consistencia:** canales y cohortes cuadran mes por mes en leads y entregas.

Decisión mía: no le creí al reporte calendario para las tasas, y separé “el agente funciona” de “el agente filtra mal”. Son dos insights distintos con palancas distintas.

Paso 4 · Impulso: modelo para 400 entregas

Qué: qué se necesita, cuánto cuesta y en qué mes es realista.

Cómo: modelo hacia atrás desde la meta, con tasas de cohortes cerradas. Está en `analysis/vemo_model.py` y en la pestaña “Impulso · Meta 400” del Sheet, con fórmulas y supuestos editables.

Paso	Cálculo	Resultado
Leads	400 ÷ 6.68 %	~5,990 al mes
Calificados	400 ÷ 48.6 %	823
Aprobación con filtro de requisitos	calificados ÷ (envíos − exceso de riesgo)	49.8 %
Análisis	823 ÷ 49.8 %	1,652 → **+7 analistas** (700 + 7 × 150)
Medios	leads actuales × CPL + leads extra × CPL × 1.5	**$1.75 M** (CAC $4.4 k, debajo del tope)
Costo del mes	medios + analistas + envíos + agente	**$2.32 M**
Margen	400 × $30 k − costo vs el margen neto de hoy	**+$6.1 M/mes**

Supuesto clave: el CPL de los leads adicionales cuesta 50 % más, porque no hay datos de cómo cambia el CPL al escalar. Lo marqué como lo primero que hay que medir.

Calendario: inventario a 60 días, contratación de analistas en 6 semanas y ~31 días de lead a entrega:

- **Nov:** filtro en el agente, medios al nivel de junio y pedido de inventario.
- **Dic:** contratar analistas.
- **Ene:** escalar medios.
- **Feb:** ~300 entregas.
- **Mar:** 400.

Respuesta: “400 en un mes” no es viable desde hoy, pero sí en marzo 2027.

Paso 5 · Conduce: retención y cobertura

Qué: probar las dos hipótesis, “no nos conocen en el sur” (Marketing) y “se van porque ganan poco” (Operaciones).

Cómo:

1. **Cobertura por hub y mes** (`conduce.v_cobertura_mensual`):
   - Centro, Poniente y Norte están en ~93 %, sobre la meta.
   - Sur A está en 79 % y Sur B en 83 % en octubre.
   - **Brecha: 785 turnos/mes × $1,100 = $0.86 M/mes.**
2. **Funnel de reclutamiento por hub** (`conduce.v_funnel_hub`):
   - El sur sí tiene leads: Sur A tiene 3,882 vs 3,915 de Centro. **La hipótesis de Marketing no se sostiene.**

- El sur pierde en la asistencia a inducción: 46 % vs 64 %.

3. **Retención a 90 días** (`conduce.v_retencion90`):
   - Solo firmas con 90 días de observación (≤ 6-ago).
   - Sin las 43 bajas por cierre de Hub Roma: la retención de Roma salía en 1 %, un valor imposible frente al ~40 % de los demás.

```
SELECT zona_hub, banda_distancia, count(*), avg(retenido_90d::int)
FROM conduce.v_retencion90 GROUP BY 1, 2;
```

   - **Por distancia:** Sur <10 km retiene **46 %** y Sur 10+ km retiene **31 %**. **80 %** de las firmas del sur vive a 10+ km, contra 40 % en los demás hubs.
   - **Cuándo se van: 55 %** de las bajas ocurre en ≤4 semanas, antes de que el bono pese.
4. **Bono vs horas.** La correlación cruda dice que más bono = más retención (34 % → 43 %). Pero el bono se paga por horas. Dentro de la misma banda (42–48 h), bono alto y bono bajo retienen igual: **36.7 % vs 36.8 %**. **La hipótesis de Operaciones no se sostiene.** Además, ~70 % del gasto del +30 % de bono ($1.36 M) iría a hubs que ya cumplen la meta.
5. **Calidad de datos:**
   - 84 firmas “Sin canal” no existen en el funnel (por eso Centro tiene 623 firmas vs 539).
   - Sur B aparece con una semana sin cobertura antes de abrir; la excluí.
   - Las firmas solo cubren 2026.

Propuesta y valor:

- **Partidas ($1.5 M en total):**
  - reclutamiento a ≤10 km ($360 k);
  - bono de referido ($200 k);
  - bono de permanencia en las semanas 4 y 12 ($330 k);
  - apoyo para asistir a inducción ($180 k);
  - **$430 k de reserva** que solo se libera si la cobertura del sur sube ≥3 pp en enero.
- **Modelo:**
  - la mezcla de firmas cercanas pasa de 20 % a 50 %;
  - la asistencia a inducción pasa de 46 % a 55 %;
  - rampa de 30/65/100 % en Q1;
  - tope en la meta de 92 %.
- **Valor:** $0.9 M (escenario bajo: cierra la mitad de la brecha) a $1.7 M (escenario base) en Q1. El modelo topa en la meta, así que presento rango y no un punto.

Paso 6 · Watts: evaluar la promoción

Qué: comprobar si las sesiones realmente se triplicaron y qué le pasó al margen.

Cómo:

1. **Sesiones de los 6 hubs de la promo:** 357 en julio y 1,174 en agosto (×3.3). Pero **4 de los 6 abrieron en jun–jul**, así que el “triple” mezcla la rampa natural de hubs nuevos con la promo.
2. **Hubs maduros vs control:**
   - D-02 y D-05 (más de 6 meses abiertos): ×1.67 en agosto contra el promedio de jul–sep. Regresaron a su nivel en septiembre.
   - Hubs destination sin promo: ×1.03.

3. **Margen** (`watts.v_hub_mes`):
   - En D-02 y D-05, 30 % de promo más 25 % de descuento LTO dejaron un precio efectivo de **$4.0/kWh, debajo del costo de energía ($5.0–5.1)**. El margen fue negativo en agosto.
   - Los 6 hubs juntos: margen de $12.7 k en agosto vs $40.3 k en septiembre.
4. **Propuesta de Q1** (`watts.v_propuesta_q1`):
   - Entran 12 hubs; **9 tienen <6 meses**, y la meta de 12 % aplica desde el sexto mes.
   - **4 venderían debajo del costo.**
   - Con descuento de 25 % y volumen ×1.7 (supuesto favorable a Marketing, sin canibalización) cuesta **$195 k de margen en Q1**; sin alza de volumen, $247 k.
   - Para empatar, los demás hubs necesitan ×2.3–7.4 los kWh, sostenido.
5. **Utilización mal medida.** La utilización usa conectores *operativos*. D-04 tiene 3 de 6 caídos desde agosto y reporta 14.9 %; sobre conectores instalados es 7.5 %. El corte de "<10 %" está midiendo mal.

Recomendación:

- Medir sobre conectores instalados y solo en hubs con ≥6 meses.
- Reparar D-04.
- Llevar la flota Vemo (Conduce + LTO) a destination en horas valle, con precio ≥ costo + $1.
- Precio por horario con piso, probado en un A/B de 4 semanas contra control.
- Renegociar el costo fijo o reubicar D-02 y D-05.

Paso 7 · Pregunta final

Comparé los tres cases por lo que está en juego, la urgencia y lo que no se sabe:

- **En juego en un trimestre:** Impulso, $23.7 M (263 entregas × $30 k × 3 meses; $7.9 M/mes); Conduce, $2.6 M (785 turnos × $1,100 × 3; $0.86 M/mes); Watts, ~$0.2 M (margen que costaría la promo en Q1).
- **Urgencia:** la de Impulso es alta, porque lo que no se pida en noviembre no llega en enero.
- **Incertidumbre:** Impulso también es el que tiene más (el CPL al escalar, la capacidad de visitas).
- **Mientras tanto:** Conduce y Watts tienen una decisión clara y de riesgo acotado. Se ejecutan con un gate a 4 semanas.

Paso 8 · Segunda vía de cálculo

Qué: que ninguna cifra del deck dependa de una sola query.

Cómo: `analysis/vemo_model.py` lee los **CSV crudos** (no la base), recalcula todo y compara contra los valores de SQL. Si algo no cuadra, el script falla. También genera:

- los JSON de los dashboards (`app/public/data/*.json`);
- `analysis/out/metrics.json`.

Resultado: 11 de 11 cifras cuadran. Una diferencia de $55/mes en Watts resultó ser redondeo de la vista SQL (precio a 2 decimales); la documenté y dejé una tolerancia explícita.

Paso 9 · Anexo en Google Sheets y Excel

Qué: que el evaluador pueda auditar y mover los supuestos sin leer código.

Cómo: `tools/gsheets/build_vemo_sheet.py` construye el Sheet y el xlsx desde los CSV:

- **Fórmulas en lugar de valores pegados:** tasas, SUMIFS por canal, COUNTIFS de retención sobre las 2,245 firmas, cobertura por hub y los modelos.
- **Supuestos editables:** van en azul sobre amarillo; todo se recalcula al cambiarlos.
- **Pestañas de datos:** "Datos · …" con las tablas originales.
- **Control de cifras:** cada cifra calculada en SQL, en Python y en el propio Sheet.

Verificación: al terminar, el script lee el Sheet de vuelta. Confirma 0 errores de fórmula y que las cifras del Sheet coinciden con Python.

Paso 10 · Deck, dashboards y app

Cómo:

1. **App:** React 19 + Vite + Tailwind en `app/`.
   - Las láminas se escriben como datos en `app/src/deck/slides.ts` y se renderizan a 1280×720.
   - Las gráficas son SVG propios.
   - Los PDF salen con Playwright (`tools/pdf/build_deck_pdfs.mjs`).
2. **Dashboards por case:** `#/dashboard/impulso|conduce|watts`, alimentados por los JSON del paso 8.
3. **Anexo:** `#/anexo` embebe el Sheet y enlaza las queries y el código en GitHub.
4. **Revisión visual:** revisé cada PDF y capturas de los dashboards.
   - Valores de mayo de un chart escritos de memoria → los reemplacé con la query.
   - "4610 %" por un formato de porcentaje → corregido.
   - Eje en 200 % → corregido.
   - Texto de "≥3×" que no correspondía a la tabla → corregido.
5. **Despliegue:** Vercel, en https://vemo-business-case.vercel.app.

Qué delegué a la IA y qué decidí yo

La IA ejecutó	Yo decidí
Carga de datos, conteos y verificación celda por celda	Usar cohortes cerradas y no el reporte calendario
Queries exploratorias y vistas SQL	Separar hubs maduros vs nuevos y armar un grupo control en Watts
Script de segunda vía y modelos	Controlar el bono por horas antes de creerle a la correlación
Scaffold de la app, export a PDF y Sheet	Excluir el cierre de Roma y la semana previa a la apertura de Sur B
Investigación pública y resumen de fuentes	Los supuestos de escala, presentar rangos y las tres recomendaciones

Errores encontrados y corregidos

#	Error	Cómo lo detecté	Corrección
1	Watts: SQL y Python difieren en $55/mes	Chequeo cruzado automático	Redondeo de la vista; documentado con tolerancia
2	Un chequeo de Conduce siempre pasaba (se comparaba consigo mismo)	Revisé el código	Lo comparo contra el valor de SQL (1,531)
3	Retención de Hub Roma de 1 %	Valor imposible	Excluí las bajas por cierre de hub
4	“Más bono = más retención”	El bono se paga por horas	Comparación dentro de la misma banda de horas
5	“La promo triplicó sesiones”	4 de 6 hubs eran nuevos	Hubs maduros vs control
6	D-04 con 14.9 % de utilización	Conectores operativos 3/6	Utilización sobre conectores instalados
7	Valores de un chart escritos de memoria	Revisión de cifras	Tomados de `watts.v_hub_mes`
8	Porcentajes ×100 y eje en 200 %	Revisión visual	Formato y escala corregidos
9	Modelo de Conduce topado en la meta	Sensibilidad	Rango y reserva condicionada
10	Cifras contradictorias en la web de Vemo	Fuentes cruzadas	Decido solo con los datos del caso
11	El script del Sheet tronaba al correrlo dos veces	Lo volví a correr	Ahora borra y recrea las pestañas

Qué no se puede saber con estos datos

- El CPL real al triplicar la inversión en Impulso.
- La capacidad de visitas y entregas físicas de Impulso.
- Por qué renuncian los conductores (no hay encuesta): la distancia es correlación. El piloto con gate lo mide.
- El efecto causal del bono de permanencia.
- Sesiones por segmento (B2C, DiDi, LTO) en Watts y la canibalización entre hubs.

Cómo reproducir todo

```
./scripts/setup_postgres.sh                                    # base + carga
tools/data_prep/.venv/bin/python -I scripts/verify_xlsx_vs_db.py   # Excel vs Postgres
docker exec -i vemo-postgres psql -U postgres -d vemo < sql/03_analytical_views.sql
tools/data_prep/.venv/bin/python -I analysis/vemo_model.py        # segunda vía + JSON
VEMO_SHEET_ID=<id> <venv-gsheets>/bin/python -I tools/gsheets/build_vemo_sheet.py   # Sheet + xlsx
(cd app && npm run build) && node tools/pdf/build_deck_pdfs.mjs   # decks en PDF
(cd app && npm run deploy)                                      # Vercel
```